

智能体动态记忆管理

Agent Dynamic Memory Management

基于 LoCoMo 的结构化记忆构建与动态状态解析 Web 应用

SRE小组 • Team ID: 第六组

成员: 王嘉淦 李胡诚 叶泽晨 马旻瑄 陈镇灏

课程: Software Requirements Engineering

2026-10

Vision (愿景)

**让 AI 智能体在长程、多会话对话中，
「记住的是最新事实，而不是被过时记忆误导」。**

一句话愿景：把「静态向量检索」升级为可追溯、可解析状态演化的动态记忆系统——不仅召回相关历史，还能判断哪些记忆已过时、被替代或相互冲突，从而给出正确回答。

① 结构化记忆构建

从 LoCoMo 对话提取人物 / 事件 / 时间 / 状态，生成结构化 Memory，支持统一 JSON 导出。

② 动态状态解析

判断记忆间 RETAIN / UPDATE / SUPERSEDE / CONFLICT 关系，定位当前有效记忆。

③ 有效记忆回答

基于当前有效 Memory 生成答案，并在 Web 端展示记忆、证据与解析过程。

要解决什么问题-Problem & Motivation

① 问题

长程、多轮对话中，用户状态、计划和偏好持续变化。普通向量检索只能「找到相关」，**无法判断「是否仍有效」**——会同时召回当前状态与被后续信息更新掉的旧状态，导致错误回答或决策。

③ 目标用户与场景

长程对话 Agent 的开发者与研究者；
跨会话「我是谁 / 我计划什么 / 我当前什么状态」类问题（长期用户画像、连续项目跟进、角色扮演长线剧情）。

② 例子

Gina 1月失业于 DoorDash → 3月开了网店。
静态检索可能把过时职业状态当当前事实；本系统应识别 **SUPERSEDE** 关系，以最新状态作答。

④ 数据与测试基础

LoCoMo长期多会话对话基准，含 QA、证据轮次、事件摘要标注。课程统一筛选「状态变化」类问题并标注相关 Memory、关系与最终有效 Memory，供系统自动检索、判断与解析。

项目范围是什么 (Scope)

In-Scope (必做)

1. 数据与记忆构建

使用 6 段 LoCoMo (4 段开发 + 2 段测试), 从原始对话提取人物/事件/时间/状态, 构建结构化 Memory (内容、时间、来源会话/轮次), 支持统一 JSON 导出, 并实现 Web 端可视化。

2. 动态维护与查询

针对输入 QA 检索相关历史 Memory, 判断 RETAIN (保留) / UPDATE (更新) / SUPERSEDE (替代) / CONFLICT (冲突), 确定当前有效 Memory 并完成回答; Web 中展示相关证据与状态解析过程。

交付: 可运行 Web 应用 + 完整源代码 + 结构化 Memory JSON + 测试结果 + 项目报告

Bonus

- 复杂冲突消解与多版本保留
- Graph Memory / 多跳关系建模
- Memory 合并与压缩
- 自适应检索 / 复杂时间推理

Out-of-Scope

不训练大模型、不构建通用对话系统、不接入真实生产环境。

准备实现哪些功能（功能清单）

功能	说明	优先级
结构化记忆构建	提取人物/事件/时间/状态 → 结构化 Memory (ID · Content · Entity/Event · Timestamp · Source · State)	必做
统一 JSON 导出	所有 Memory 导出为统一 JSON 格式	必做
Web 记忆可视化	查看结构化记忆 + 原始证据（会话/轮次）	必做
记忆检索	给定问题，从长期历史检索相关 Memory	必做
状态关系判定	判断 RETAIN / UPDATE / SUPERSEDE / CONFLICT	必做
有效记忆解析 + 回答	确定当前有效 Memory，据此生成答案	必做
Web 状态解析展示	展示相关历史 Memory 及其关系、解析过程	必做
冲突消解 / 多版本保留	更复杂的冲突消解与多版本保留	Bonus
Graph Memory	图记忆 / 多跳关系建模	Bonus
记忆合并压缩	Memory 合并与压缩	Bonus
自适应检索 / 时间推理	自适应检索或复杂时间推理	Bonus

建议 Memory 字段：Memory ID · Content · Entity/Event · Timestamp · Source Session/Turn · Query Relation · Validity/State

系统概述 (Pipeline) 与状态关系示例

① 长对话 (多会话)

LoCoMo 多会话对话数据



② 结构化记忆抽取

Memory: ID · Content · Entity/Event · Timestamp · Source · State



③ 问题驱动检索

相关候选 Memory



④ 状态关系判定

RETAIN / UPDATE / SUPERSEDE / CONFLICT



⑤ 有效 Memory 解析

LLM 生成答案 + 证据展示 (Web)

状态关系示例 (真实 LoCoMo)

UPDATE 更新

Caroline: researching → applied → passed adoption. 最新阶段为当前有效状态, 前序保留为历史。

SUPERSEDE 替代

Gina: 1月失业 DoorDash → 3月网店开业。旧状态被新状态替代, 历史事实仍可追溯。

CONFLICT 冲突

Gina 已失业 vs 仍在 DoorDash 工作。两条当前事实不可同时成立且无充分证据消解 → 标记冲突、暂并存并保留来源。

Feature Roadmap (版本路线图)

版本	里程碑	功能
V1 (MVP)	M3	数据选段 (4 dev + 2 test) · 结构化记忆构建 · 统一 JSON 导出 · Web 记忆/证据可视化
V2 (核心)	M4	记忆检索 · 状态关系判定 (RETAIN/UPDATE/SUPERSEDE/CONFLICT) · 有效记忆解析 + QA · Web 状态解析展示
V3 (增强)	M5	Bonus完成- (Graph 记忆 / 冲突消解 / 自适应检索) · 完整对照评测 · 报告与终版展示

推进方式:

每个版本对应一个里程碑，同版本至少两轮迭代（初稿，评审，修订定稿），截止前预留 1-2 天 buffer，保证功能逐层递进、可追溯。

在M4之前完成in-scope内容，最终版本之前完成所有bonus内容

数据与评测

数据来源

从官方 10 段长对话选 6 段：**4 段 dev + 2 段 test**（其余 4 段作扩展评测）。每段平均 588.2 轮、27.2 个会话、约 1.66 万 Token，含 QA / 证据轮次 / 事件摘要标注。

数据与代码：github.com/snap-research/locomo

动态记忆测试

课程从所选 LoCoMo QA 中统一筛选「状态变化」问题，并标注相关历史 Memory、Memory 间的保留 / 更新替代关系及最终有效 Memory。系统需自动检索相关 Memory、完成关系判断与状态解析后再生成答案。

评测指标与对照

评测：问题回答 F1 · 证据记忆检索 Recall@K · 动态状态更新判断准确率。

对照 (Baseline)：Full Text (全文) · Vector RAG (检索相关 Memory 但不做状态关系判断)。

参考文献

Maharana A, et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. ACL 2024.

后续如何推进（计划）

里程碑	内容	占比	截止
M1	Team Workflow	15%	10/10
M2	Project Proposal	5%	10/10
M3	SRS	15%	10/24
M4	Mid-term（设计/编码/测试）	35%	10/24
M5	Final Presentation & Report	30%	11/9

推进方式

- 每里程碑至少两轮迭代（初稿 → 评审 → 修订定稿），截止前预留 1-2 天 buffer。
 - 阶段一（10/8-10/10）M1+M2；阶段二（10/11-10/24）M3+M4；阶段三（10/25-11/9）M5。
- 阶段二之前完成inscope内容，阶段三之前完成bonus,并集成完毕。

分工

组长/项目经理：**王嘉淦**
需求/数据+算法：**李胡诚**
后端开发：**叶泽晨**
前端开发：**马旻瑄**
测试/DevOps+评测：**陈镇灏**

下一步行动

- 已完成：建仓库、process/ 目录、完成会议纪要模板、完成首次会议
- 接下来：选定 4 dev + 2 test 数据、设计 Memory 字段与 JSON 结构、SRS v0.1。

成功标准 / 交付与评分 / 风险

交付内容

- ① 可运行的 Web 应用
- ② 完整源代码
- ③ 结构化 Memory JSON
- ④ 实验结果与对照分析
- ⑤ 项目报告与课堂展示

严格按照评分依据完成项目

- ① 各里程碑交付物质量
- ② 版本控制
- ③ 项目管理
- ④ 中期与期末展示

风险与对策

数据量大（每段 ~588 轮）：用 4 段 dev 尽早跑通流水线，再上 2 段 test 复测。

时间紧、迭代多：每里程碑两轮迭代 + 提前 1-2 天 buffer；功能逐版本递进、可追溯。